

Lecture #6: High-performance kernel using Triton

python kernel; cuda kernel; C++ kernel; Triton kernel;

Last lecture: high-level overview of GPUs and performance

This lecture: benchmarking / profiling + write kernels
↳ how long does it take? where time is spent?

↓

write kernels: python using triton.

write kernels: C++ / ~~using~~ cuda.

5 ways to write a function: manual, pytorch, compiled,
CUDA, Triton

ReLU (element-wise), softmax (row-wise), matmul (complex aggregation)

key principle: organize computation to minimize reads/write.

key ideas: kernel fusion (warehouse / factory analogy), tiling (shared memory)
Auto-compilers (Triton, torch.compile) will get better over time

execution model:

thread: process individual index (i.e. $f(i)$)

Thread block: (concurrent thread arrays): scheduled on a single SM

Grid: collection of thread blocks

why thread blocks: shared memory

⇒ intuition: group threads that read similar data together

⇒ threads within a thread block have shared memory

⌞ (as fast as L1 cache) (A100: 164KB)

⇒ can synchronize/synchronise threads for writing/reading within a block (not across blocks)

Arithmetic intensity: #Flops/#bytes

⇒ if high, operation is compute-bound (good)

⇒ if low, operation is memory-bound (bad)

General rule: matrix multiplication is compute-bound, everything else is memory-bound

benchmarking: how long does it take? (compare which faster)
profiling: where time is being spent? (how perf scales with dim)

⇒ whenever make a change on operator,

do benchmarking / profiling

demo benchmarking benchmarking square matrix multiplication

Size	Timing
1024	→ 0.871
2048	→ 0.839
4096	→ 3.176
8192	→ 21.886
16384	→ 162.935

Benchmark simple MLP ⇒ scaling

⇒ scale num of steps (runtime) ← linear scaling

⇒ scale num of layer ⇒ scale batch size; scale the dimension